

Lecture 09: Sampling-based Motion Planning

Dr. Sihui Li

Washington State University Tri-Cities

Spring 2025



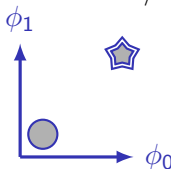
WASHINGTON STATE UNIVERSITY
TRI-CITIES

Learning Outcome

- Understand the motion planning problem definition.
- Understand the key components of a sampling-based motion planner.
- Apply sampling-based algorithms such as RRT and PRM.

Overview

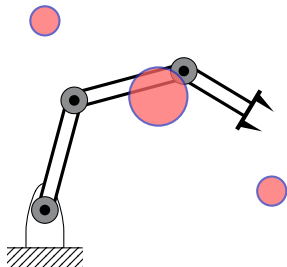
- 1 Start from initial configuration
- 2 Sample new configurations
- 3 **Construct** and **Add** valid states to tree/graph
- 4 Terminate when:
 - ▶ We find a path to the goal
 - ▶ We exhaust a timeout / max samples



Outline

- 1 Sampling-Based Motion Planning
- 2 Metric Spaces
- 3 Rapidly-Exploring Random Trees (RRT)
- 4 Probabilistic Roadmaps (PRM)

Configuration Space



- Configuration Space $\mathcal{Q}$
- Obstacle Region: $\mathcal{Q}_{\text{obs}} = \{q \in \mathcal{Q} \mid \neg \text{is-valid}(q)\}$
- Free Space: $\mathcal{Q}_{\text{free}} = \{q \in \mathcal{Q} \mid \text{is-valid}(q)\}$
- Total Space:
 - ▶ $\mathcal{Q}_{\text{obs}} \cap \mathcal{Q}_{\text{free}} = \emptyset$
 - ▶ $\mathcal{Q}_{\text{obs}} \cup \mathcal{Q}_{\text{free}} = \mathcal{Q}$

Generally: no explicit representation of $\mathcal{Q}_{\text{obs}}$, $\mathcal{Q}_{\text{free}}$

Example

`https://github.com/ethz-asl/amr\_visualisations?tab=readme-ov-file`

Piano Mover's Problem

Definition

Piano Mover's Problem

- **Given:** Environment, Robot, Start and Goal configurations
 - ▶ **World** $\mathcal{W}$ in $\mathbb{R}^2$ or $\mathbb{R}^3$
 - ▶ **Robot** in $\mathcal{W}$: either a single or collection of rigid bodies
 - ▶ **Configuration space** $\mathcal{Q}$ for the robot
 - ▶ **Initial configuration** $q_0 \in \mathcal{Q}_{\text{free}}$
 - ▶ **Goal configuration** $q_G \in \mathcal{Q}_{\text{free}}$
- **Find:** Valid path from q_0 to q_G .

What data structure for “world” and “robot?”

We can define flexible abstractions.

Piano Mover's Problem

Definition

Piano Mover's Problem

- **Given:** Environment, Robot, Start and Goal configurations
 - ▶ **World** $\mathcal{W}$ in $\mathbb{R}^2$ or $\mathbb{R}^3$
 - ▶ **Robot** in $\mathcal{W}$: either a single or collection of rigid bodies
 - ▶ **Configuration space** $\mathcal{Q}$ for the robot
 - ▶ **Initial configuration** $q_0 \in \mathcal{Q}_{\text{free}}$
 - ▶ **Goal configuration** $q_G \in \mathcal{Q}_{\text{free}}$
- **Find:** Valid path from q_0 to q_G .

What data structure for “world” and “robot?”

We can define flexible abstractions.

Piano Mover's Problem

Definition

Piano Mover's Problem

- **Given:** Environment, Robot, Start and Goal configurations
 - ▶ **World** $\mathcal{W}$ in $\mathbb{R}^2$ or $\mathbb{R}^3$
 - ▶ **Robot** in $\mathcal{W}$: either a single or collection of rigid bodies
 - ▶ **Configuration space** $\mathcal{Q}$ for the robot
 - ▶ **Initial configuration** $q_0 \in \mathcal{Q}_{\text{free}}$
 - ▶ **Goal configuration** $q_G \in \mathcal{Q}_{\text{free}}$
- **Find:** Valid path from q_0 to q_G .

What data structure for “world” and “robot?”

We can define flexible abstractions.

Fundamental Abstractions

Configuration Space: Set of possible configurations

- $\mathcal{Q}$

Configuration Sampler: Generates candidate configurations

- $() \mapsto \mathcal{Q}$

Validity Checker: Is some configuration in $\mathcal{Q}_{\text{free}}$?

- $\underbrace{\mathcal{Q}}_{\text{sample}} \mapsto \underbrace{\{0, 1\}}_{\text{valid?}}$

Nearest Neighbors: Closest point in tree/graph to new sample:

- $\underbrace{\mathcal{P}(\mathcal{Q})}_{\text{tree}} \times \underbrace{\mathcal{Q}}_{\text{new sample}} \mapsto \underbrace{\mathcal{Q}}_{\text{nearest neighbor}}$

- **Distance metric:** $\mathcal{Q} \times \mathcal{Q} \mapsto \mathbb{R}$

Sampling-based motion planning generalizes beyond robotics.

Motion Planning Problem, Redux

Definition

Piano Mover's Problem

- **Given:** Environment, Robot, Start and Goal configurations
 - ▶ **World** $\mathcal{W}$ in $\mathbb{R}^2$ or $\mathbb{R}^3$
 - ▶ **Robot** in $\mathcal{W}$: either a single or collection of rigid bodies
 - ▶ **Configuration space** $\mathcal{Q}$ for the robot
 - ▶ **Initial configuration** $q_0 \in \mathcal{Q}_{\text{free}}$
 - ▶ **Goal configuration** $q_G \in \mathcal{Q}_{\text{free}}$
- **Find:** Valid path from q_0 to q_G .

Definition

Sampling-based Motion Planning

- **Given:** Metric state space, validity checker, start, goal
 - ▶ $\mathcal{Q}$, state space
 - Metric function: $\mathcal{Q} \times \mathcal{Q} \mapsto \mathbb{R}$
 - (Uniform) Sampler: $q \sim \mathcal{Q}$
 - ▶ $v : \mathcal{Q} \mapsto \{0, 1\}$, validity checker
 - ▶ $q_0 \in \mathcal{Q}$, initial state
 - ▶ $q_g \sim G$, goal sampler
- **Find:** Valid path from q_0 to a q_g .

Motion Planning Problem, Redux

Definition

Piano Mover's Problem

- **Given:** Environment, Robot, Start and Goal configurations
 - ▶ **World** $\mathcal{W}$ in $\mathbb{R}^2$ or $\mathbb{R}^3$
 - ▶ **Robot** in $\mathcal{W}$: either a single or collection of rigid bodies
 - ▶ **Configuration space** $\mathcal{Q}$ for the robot
 - ▶ **Initial configuration** $q_0 \in \mathcal{Q}_{\text{free}}$
 - ▶ **Goal configuration** $q_G \in \mathcal{Q}_{\text{free}}$
- **Find:** Valid path from q_0 to q_G .

Definition

Sampling-based Motion Planning

- **Given:** Metric state space, validity checker, start, goal
 - ▶ $\mathcal{Q}$, state space
 - ★ Metric function: $\mathcal{Q} \times \mathcal{Q} \mapsto \mathbb{R}$
 - ★ (Uniform) Sampler: $q \sim \mathcal{Q}$
 - ▶ $v : \mathcal{Q} \mapsto \{0, 1\}$, validity checker
 - ▶ $q_0 \in \mathcal{Q}$, initial state
 - ▶ $q_g \sim G$, goal sampler
- **Find:** Valid path from q_0 to a q_g .

Outline

- 1 Sampling-Based Motion Planning
- 2 Metric Spaces**
- 3 Rapidly-Exploring Random Trees (RRT)
- 4 Probabilistic Roadmaps (PRM)

Definition: Metric Space

Definition: Metric Space

A **metric space** is a space $\mathcal{Q}$ equipped with a function $\rho : \mathcal{Q} \times \mathcal{Q} \mapsto \mathbb{R}$ that has the following properties for any $a, b, c \in \mathcal{Q}$

Nonnegative: $\rho(a, b) \geq 0$

(Distances are always greater than zero)

Reflexive: $(\rho(a, b) = 0) \iff (a = b)$

(Distance is zero only for identical elements)

Symmetric: $\rho(a, b) = \rho(b, a)$

(Distance from a to b is the same as from b to a)

Triangle: $\rho(a, b) + \rho(b, c) \geq \rho(a, c)$

(a to b to c cannot be shorter than directly from a to c)

Example: Robot Joint Space and L_p metrics

$$\rho(x, x') = \left(\sum_{i=0}^{n-1} |x_i - x'_i|^p \right)^{\frac{1}{p}}$$

- **Space:**

$\mathcal{Q} \subseteq \mathbb{R}^n$, the set of robot joint positions

- **Euclidean Distance:**

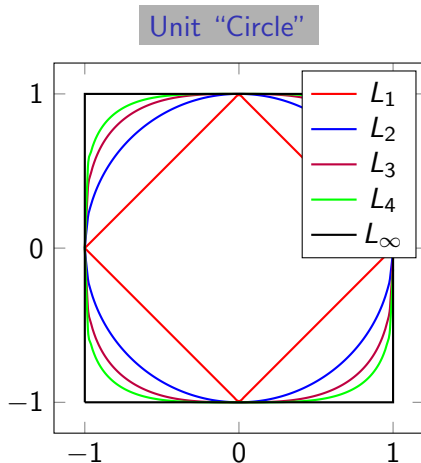
$$L_2(x, x') = \sqrt{\sum_{i=0}^{n-1} (x_i - x'_i)^2}$$

- **Manhattan Distance:**

$$L_1(x, x') = \sum_{i=0}^{n-1} |x_i - x'_i|$$

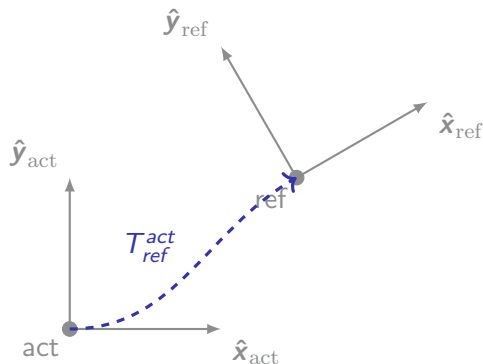
- **L_∞ :**

$$L_\infty(x, x') = \max_{0 \leq i \leq n-1} (|x_i - x'_i|)$$



Example: Transformation Metric

- How far are we from the desired pose (position and orientation)?
- Define a distance metric between two transformation matrices.
 - ▶ Position: Euclidean Distance.
 - ▶ Rotation: Angle of rotation between two rotation matrices.
 - ▶ Combine the two with constant factors.



Exercise

Given ρ_1 and ρ_2 are both metric functions for some metric space Q . Prove that $c_1\rho_1 + c_2\rho_2$ is also a metric function for Q . c_1 and c_2 are positive constants. Is it still true when c_1 and c_2 are negative constants?

Outline

- 1 Sampling-Based Motion Planning
- 2 Metric Spaces
- 3 Rapidly-Exploring Random Trees (RRT)**
- 4 Probabilistic Roadmaps (PRM)

RRT Illustration

https://lavalle.pl/rrt/gallery_2drrt.html

RRT Algorithm

Algorithm 1: Rapidly-Exploring Random Tree

Input: $\phi_0 \in \mathcal{Q}$: start

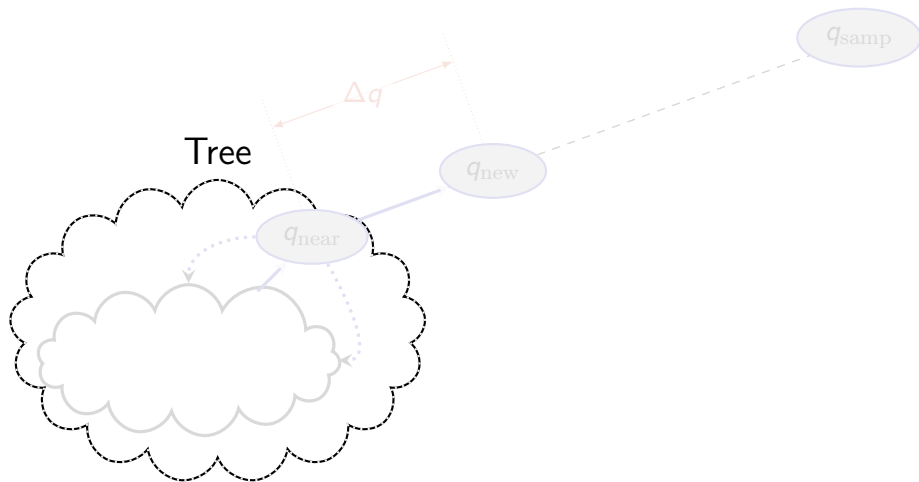
Input: $\Phi_g \subseteq \mathcal{Q}$: goal set

```
1  $V \leftarrow \{\phi_0\}$ ; // Tree nodes
2  $E \leftarrow \emptyset$ ; // Tree edges
3 for  $k \leftarrow 0$  to LIMIT do
4    $q_{\text{samp}} \leftarrow \text{sample}()$ ;
5    $q_{\text{near}} \leftarrow \text{nearest-neighbor}(V, q_{\text{samp}})$ ;
6    $q_{\text{new}} \leftarrow \text{new-conf}(q_{\text{near}}, q_{\text{samp}})$ ;
7   if  $\text{valid}(q_{\text{new}})$  then
8     if  $\exists q \in \Phi_g, \text{dist}(q, q_{\text{new}}) < \epsilon$  then
9       return path from  $q_0$  to  $q_{\text{new}}$ ;
10     $V \leftarrow V \cup \{q_{\text{new}}\}$ ;
11     $E \leftarrow E \cup \{q_{\text{near}} \rightarrow q_{\text{new}}\}$ ;
12 return TIMEOUT;
```

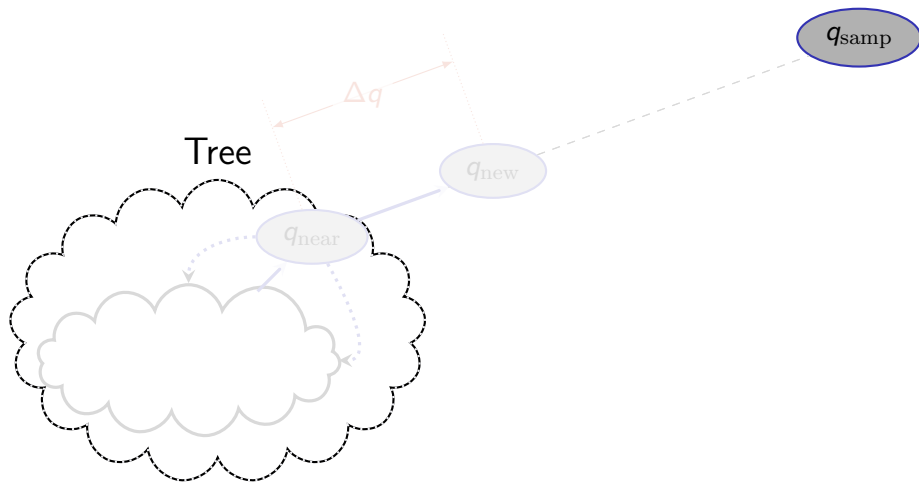
Subroutines

- sample
- nearest-neighbor
- new-conf
- valid
- dist

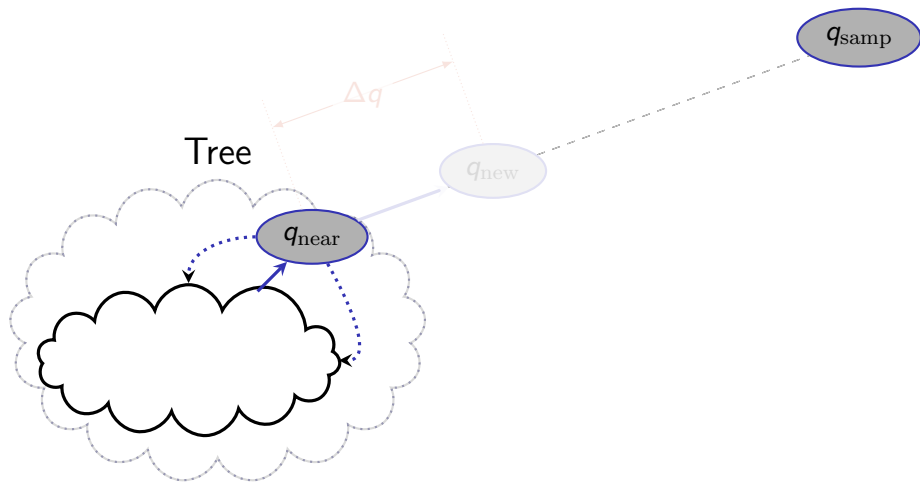
RRT Step Illustration



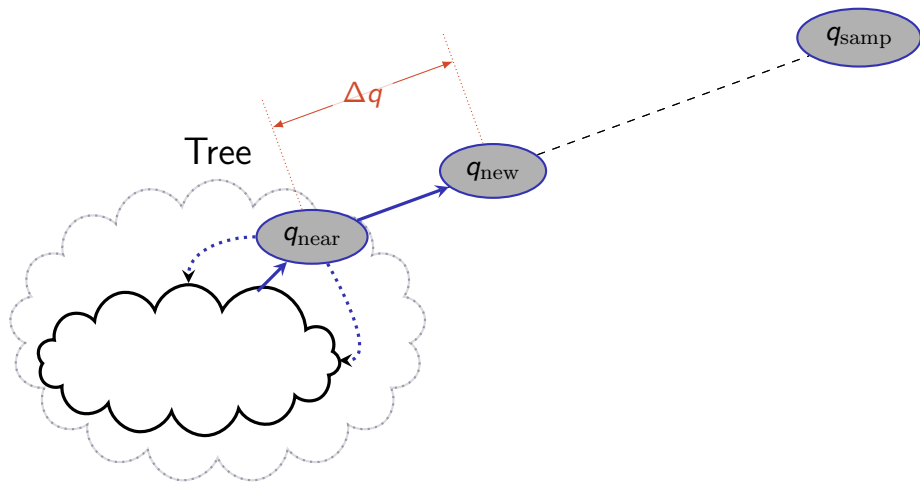
RRT Step Illustration



RRT Step Illustration

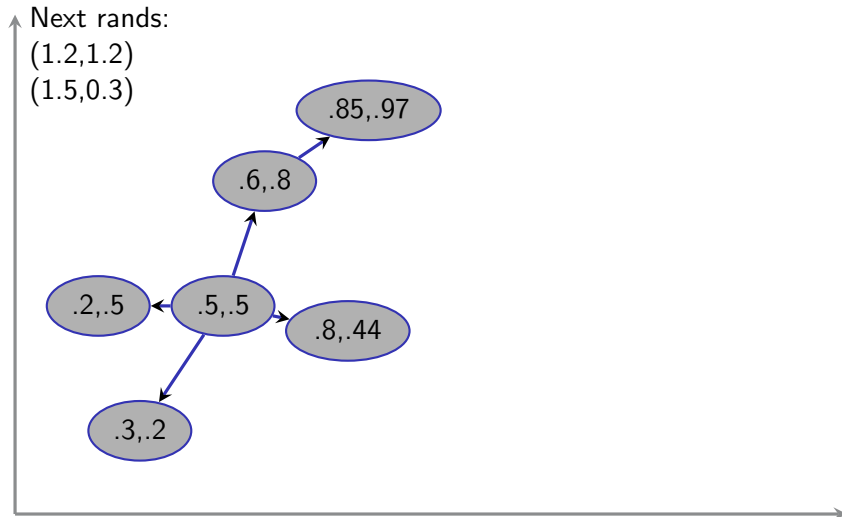


RRT Step Illustration



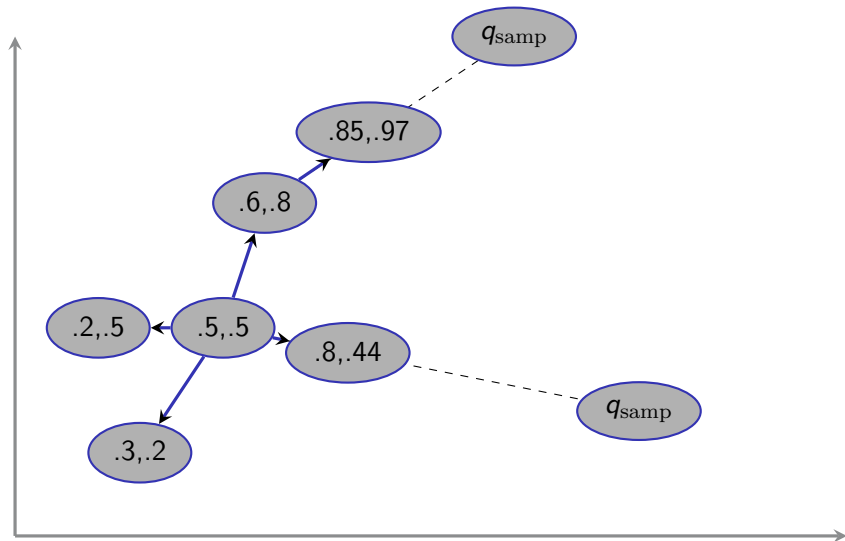
Exercise: RRT-Step

Extend the RRT. Use $(1.2, 1.2)$ and $(1.5, .3)$ as the next random points.



Exercise: RRT-Step

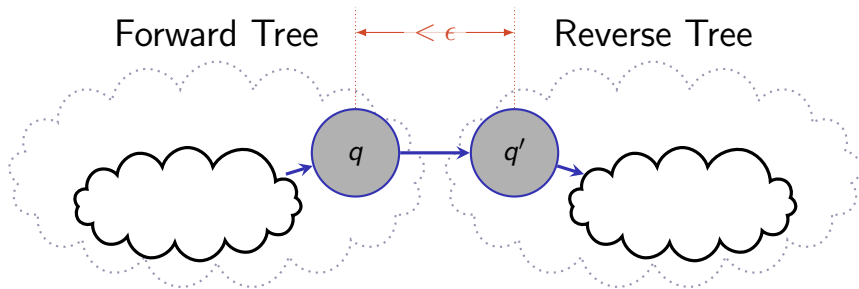
Extend the RRT. Use $(1.2, 1.2)$ and $(1.5, .3)$ as the next random points.



RRT-Connect

Bi-directional RRT

- 1 Construct one RRT, rooted at the start
- 2 Construct second RRT, rooted at the goal
- 3 Terminate when the two trees connect



Outline

- 1 Sampling-Based Motion Planning
- 2 Metric Spaces
- 3 Rapidly-Exploring Random Trees (RRT)
- 4 Probabilistic Roadmaps (PRM)

PRM Overview

Preprocessing: Construct the Roadmap

- ① Sample new configurations
- ② Connect to “neighboring” configurations in roadmap

Query: Search the Roadmap

- ① Find path in roadmap via discrete search (e.g., A^*)
- ② Connect subsequent configurations with “local planner” (interpolate)

PRM Construction

Algorithm

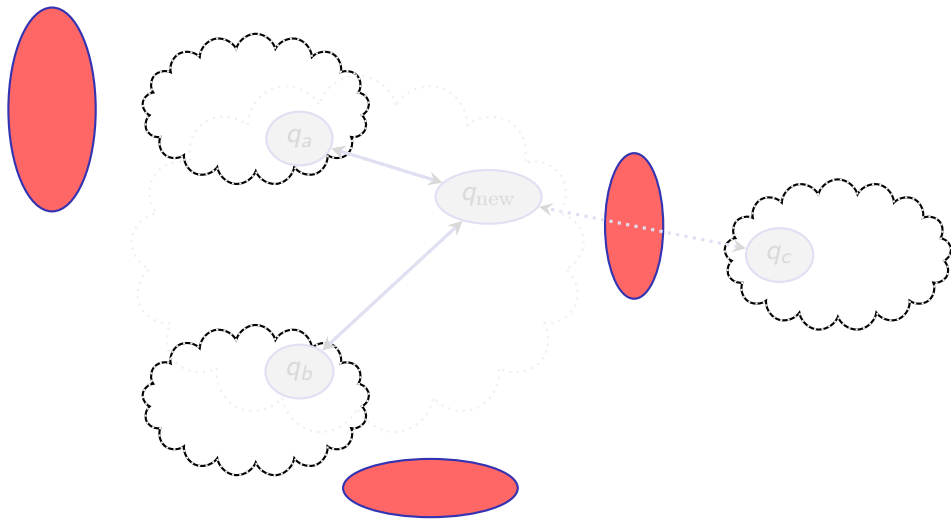
Algorithm 2: Construct Probabilistic Roadmap

```
1  $V \leftarrow \emptyset$ ; // Roadmap nodes
2  $E \leftarrow \emptyset$ ; // Roadmap Edges
3  $k \leftarrow 0$ ; // Iteration count
4 while  $k < \text{LIMIT}$  do
5      $q_{\text{new}} \leftarrow \text{sample}()$ ;
6     if  $\text{valid}(q_{\text{new}})$  then
7          $V \leftarrow V \cup \{q_{\text{new}}\}$ ;
8          $k \leftarrow k + 1$ ;
9         foreach  $q \in \text{neighborhood}(q_{\text{new}})$  do
10             if  $\neg \text{connected}(q, q_{\text{new}}) \wedge$ 
11                  $\text{connect}(q, q_{\text{new}})$  then
12                  $E \leftarrow E \cup \{q \rightarrow q_{\text{new}}\}$ ;
12 return  $(V, E)$ ;
```

- Neighbors:
 - ▶ K-nearest neighbors (KNN)
 - ▶ KNN from connected-components of V
 - ▶ Radius (within ϵ distance)
- Connected Components:
 - ▶ Optional: do we want different paths?
 - ▶ Efficiency: Track as edges are added
- Connect: valid path exists
 - ▶ E.g., linearly interpolate and check validity

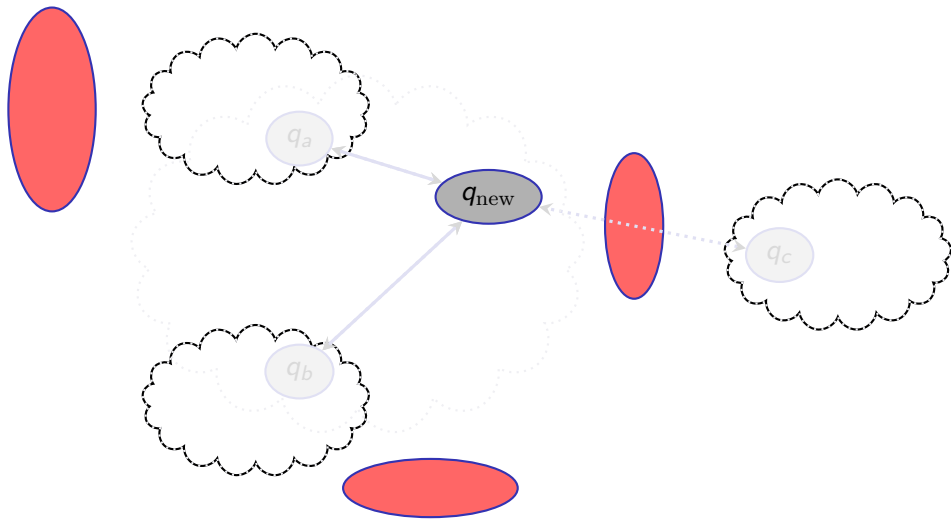
PRM Construction

Illustration



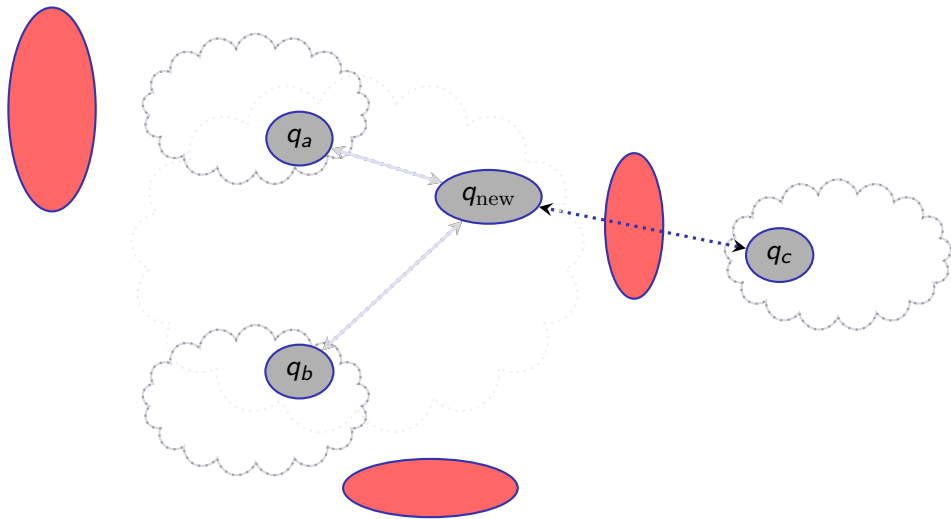
PRM Construction

Illustration



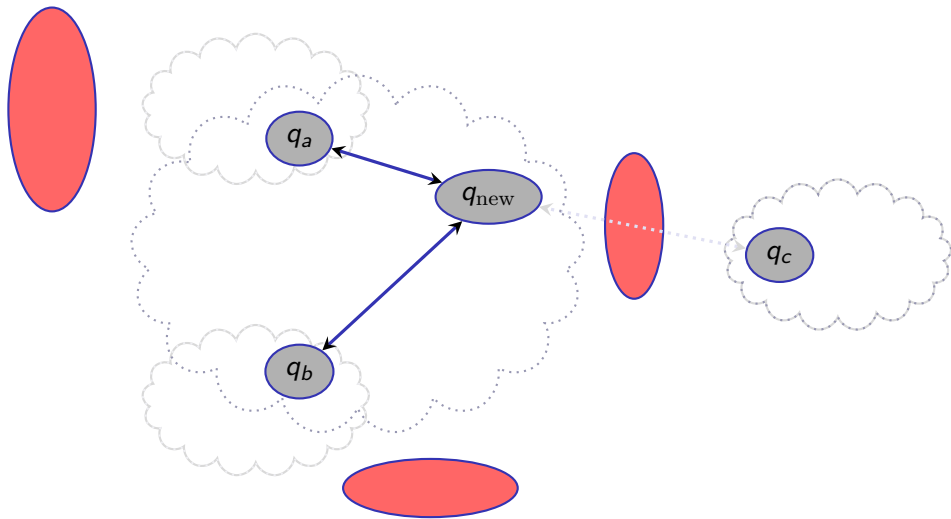
PRM Construction

Illustration



PRM Construction

Illustration



Practical Considerations

- Nearest Neighbors: Kd-tree (partitioning) or brute force (for high-dof)
- Collision Checking: Bounding-boxes then decompose into triangles
- Paths: Smooth and Shortcut
- Hundreds of variations on RRT, PRM, and similar methods
- RRT-Connect usually works pretty well

Conclusion

- Motion planning and sampling-based motion planner.